

QA Evidence Guide

Dependency Usage Analysis for Ant / TERASOLUNA Migration

Mục tiêu: lay evidence de giam scope migration dependency mot cach an toan

Context ngan gon

Project Java legacy dang dung Apache Ant. Mot so module nhu keilib va servercommon khong co build file riêng, ma duoc build chung tu module KEIKA. KEIKA/build.xml dang tro den cac thu muc thu vien shared nhu /lib/terasoluna5, /lib/test, /lib/extension. File all.userlibraries co the la Eclipse IDE configuration, nhung khong du de ket luan dependency source of truth cho tung module.

1. Ket luan chinh

Muon thuyet phuc QA hoac khach hang rang khong can migrate toan bo JAR trong TERASOLUNA/test/extension, can tao evidence theo tung lop. Khong nen chi dua vao all.userlibraries, vi file do chi cho thay Eclipse dang khai bao nhieu JAR, khong chung minh moi module that su dung tat ca cac JAR do.

Cau chot de noi voi QA

all.userlibraries shows what is available to Eclipse, but not necessarily what each module actually uses. The migration scope should be based on actual Ant build classpath plus source, bytecode, configuration, and runtime evidence.

2. Mục tiêu evidence cuối cùng

Tao duoc dependency usage matrix de tra loi cac cau hoi:

- Dependency nao that su duoc project dung?
- Dependency do duoc dung o module nao?
- Bang chung la source code, compiled class, XML/JSP/config, Ant classpath hay runtime log?
- Dependency do can migrate, chi can test scope, provided by server, hay co the exclude?

Module	Dependency/JAR	Evidence	Evidence type	Action
keilib	spring-core	xxxService.java import org.springframework...	Source import	Migrate
servercommon	jackson-databind	ObjectMapper used in JsonUtil.java	Source import	Migrate
KEIKA	mybatis	sqlMapConfig.xml / mapper XML	Config/XML	Migrate
KEIKA	junit	Only under test folder	Test only	Test scope
all	tomcat-servlet-api	Provided by app server	Runtime provided	Do not package
unknown	commons-compress	No source/jdeps/runtime evidence	Unused candidate	Do not migrate unless confirmed

3. Không nên đưa vào một loại scan duy nhất

Legacy Java thường dùng dependency qua import Java, annotation, XML Spring, JSP taglib, reflection, batch job hoặc server runtime. Vì vậy cần tách evidence thành 5 lớp:

```
Layer 1: Actual Ant build classpath
Layer 2: Source import scan
Layer 3: Bytecode scan using jdeps
Layer 4: Config / JSP / XML / properties scan
Layer 5: Runtime loaded-class log
```

4. Layer 1 - Lay classpath that sử dụng Ant / KEIKA

Bước đầu tiên là chứng minh KEIKA build đang dùng classpath nào. Đây quan trọng hơn all.userlibraries vì Ant build mới là thứ compile thành su.

Trong build.xml, tìm các path/fileset đang trỏ đến các thư mục lib shared:

```
<path id="xxx.classpath">
  <fileset dir="${lib.dir}/terasoluna5">
    <include name="*.jar"/>
  </fileset>
  <fileset dir="${lib.dir}/test">
    <include name="*.jar"/>
  </fileset>
  <fileset dir="${lib.dir}/extension">
    <include name="*.jar"/>
  </fileset>
</path>
```

Thêm target tạm để print classpath:

```
<target name="print-classpath">
  <property name="resolved.classpath" refid="compile.classpath"/>
  <echo message="=== COMPILE CLASSPATH ==="/>
  <echo message="${resolved.classpath}"/>
</target>
```

Chạy lệnh:

```
mkdir -p evidence
ant print-classpath > evidence/01-ant-classpath.txt
```

Evidence tạo ra

evidence/01-ant-classpath.txt

Dùng file này để nói với QA: This is the actual Ant build classpath used by KEIKA, not only Eclipse IDE configuration.

5. Layer 2 - Scan source import theo từng module

Mục tiêu là biết source code của keilib, servercommon và KEIKA đang import package nào.

```
mkdir -p evidence
```

```
find keilib servercommon KEIKA -name "*.java" -exec grep -H "^import " {} \; > evidence/02-source-imports.txt
```

Loc unique import package:

```
cat evidence/02-source-imports.txt | sed 's/.*import //' | sed 's/;/;' | sort -u > evidence/02-source-imports-unique.txt
```

Vi du evidence co the nhin nhu sau:

```
servercommon/src/main/java/.../JsonUtil.java:import com.fasterxml.jackson.databind.ObjectMapper;
keilib/src/main/java/.../DateUtil.java:import org.joda.time.DateTime;
KEIKA/src/main/java/.../UserMapper.java:import org.apache.ibatis.session.SqlSession;
```

Package pattern	Dependency group
org.springframework.*	Spring
org.springframework.security.*	Spring Security
com.fasterxml.jackson.*	Jackson
org.apache.ibatis.*	MyBatis
org.terasoluna.gfw.*	TERASOLUNA GFW
org.joda.time.*	Joda-Time
org.slf4j.*	SLF4J
ch.qos.logback.*	Logback
org.apache.tiles.*	Tiles
javax.servlet.*	Servlet API - usually provided by server

6. Layer 3 - Dung jdeps de scan compiled classes

Source import chua du, vi code co the dung dependency qua annotation, inherited class, method signature hoac generated class. Sau khi build xong, chay jdeps tren tung module output.

```
mkdir -p evidence/jdeps

JARS=$(find ServerLib/lib/terasoluna5 ServerLib/lib/extension -name "*.jar" | tr ' ' '\n')

jdeps -verbose:class -recursive -classpath "$JARS" keilib/build/classes > evidence/jdeps/keilib-jdeps.txt

jdeps -verbose:class -recursive -classpath "$JARS" servercommon/build/classes > evidence/jdeps/servercommon-jdeps.txt

jdeps -verbose:class -recursive -classpath "$JARS" KEIKA/build/classes > evidence/jdeps/KEIKA-jdeps.txt
```

Vi du output jdeps co the cho thay:

```
com.company.common.JsonUtil -> com.fasterxml.jackson.databind.ObjectMapper jackson-databind-2.13.1.jar
com.company.security.AuthUtil -> org.springframework.security.core.Authentication spring-security-core-5.4.2.jar
```

Gia tri QA

Day la evidence bytecode-level. Neu source scan bi thieu do annotation/inheritance/signature, jdeps co the bat duoc dependency tu compiled output.

7. Layer 4 - Scan XML / JSP / properties / config

Legacy TERASOLUNA/Spring app thường dùng dependency trong XML/JSP mà Java import không thấy. Có thể scan các file config và view.

```
find keilib servercommon KEIKA \ ( -name "*.xml" -o -name "*.properties" -o -name "*.jsp" -o -name
"*.tag" -o -nam
e "*.tld" \) -exec grep -HnE "org\.springframework|org\.terasoluna|org\.apache\.ibatis|
com\.fasterxml|javax\.
servlet|org\.apache\.tiles|org\.quartz|org\.dozer|joda|logback|slf4j" {} \; > evidence/04-config-
dependency-u
sage.txt
```

Vi du evidence:

```
KEIKA/src/main/resources/spring/applicationContext.xml:
<bean class="org.mybatis.spring.SqlSessionFactoryBean">

KEIKA/src/main/webapp/WEB-INF/views/layout.jsp:
<%@ taglib prefix="sec" uri="http://www.springframework.org/security/tags" %>
```

Dependency hay nằm trong config/JSP

spring-webmvc, spring-security-config, mybatis-spring, tiles-jsp, taglibs, dozer, quartz.

8. Layer 5 - Runtime loaded-class evidence

Đây là evidence mạnh nếu khách hàng muốn biết app chạy đã tải JAR nào. Tuy nhiên runtime evidence phụ thuộc vào test coverage.

Neu dùng Java 8:

```
java -verbose:class ...
```

Neu dùng Java 9 trở lên:

```
java -Xlog:class+load=info ...
```

Capture log khi chạy app hoặc integration test:

```
mkdir -p evidence/runtime

# Java 8 example
JAVA_OPTS="$JAVA_OPTS -verbose:class" ./run-server.sh > evidence/runtime/05-loaded-classes.log 2>&1
```

Extract JAR đã tải:

```
grep "\[Loaded" evidence/runtime/05-loaded-classes.log | grep ".jar" | sed -E 's/.*from //' |
sort -u > evi
dence/runtime/05-runtime-loaded-jars.txt
```

Lưu ý quan trọng

Nếu một màn hình, batch job hoặc external flow chưa được chạy, runtime log có thể không load JAR liên quan. Vì vậy chỉ nên viết: No evidence found in current analysis, không nên viết chắc chắn unused 100%.

9. Cách phân loại dependency de giam scope migration

Sau khi có evidence, chia dependency thành 6 nhóm:

Group	Mô tả	Action
A - Directly used by source	Có import trực tiếp trong Java source code.	Must migrate
B - Used by config/XML/JSP	Không thấy trong Java import nhưng thấy trong XML/JSP/properties.	Must migrate
C - Transitive dependency	Project không import trực tiếp nhưng dependency nhóm A/B có nó.	Keep as transitive; do not manually declare unless needed
D - Test only	Chỉ dùng trong test source hoặc /lib/test.	Test scope only
E - Provided by application server	Compile có, nhưng runtime server cung cấp.	Provided scope / do not package into WAR
F - No evidence / unused candidate	Không thấy source import, jdeps, XML/JSP, runtime log.	Candidate to exclude from migration unless confirmed

10. Chi tiết từng nhóm dependency

Group A - Directly used by source

Có import trực tiếp trong Java source. Action: Must migrate.

```
com.fasterxml.jackson.databind.ObjectMapper
org.springframework.stereotype.Service
org.apache.ibatis.session.SqlSession
```

Group B - Used by config/XML/JSP

Không thấy trong Java import nhưng thấy trong XML/JSP. Action: Must migrate.

```
spring-security-taglibs
tiles-jsp
mybatis-spring
dozer-spring4
```

Group C - Transitive dependency

Project không import trực tiếp, nhưng dependency nhóm A/B có nó. Action: Keep as transitive, không cần manually declare trừ khi build tool yêu cầu.

```
jackson-core
jackson-annotations
spring-core
spring-jcl
```

Group D - Test only

Chỉ dùng trong test source hoặc /lib/test. Không nên migrate chung với runtime TERASOLUNA nếu mục tiêu chỉ là runtime migration.

```
junit
mockito
dbunit
cobertura
```

```
poi
xstream
spring-test
```

Group E - Provided by application server

Compile can, nhưng runtime app server có the cung cấp. Có thể xác nhận app server và packaging policy trước khi include vào WAR.

```
servlet-api
jsp-api
el-api
tomcat-servlet-api
tomcat-jsp-api
tomcat-el-api
```

Group F - No evidence / unused candidate

Không thấy source import, không thấy jdeps, không thấy XML/JSP, không thấy runtime log. Action: Candidate to exclude from migration.

Wording an toàn

No evidence found in current static/runtime analysis. Recommend excluding from migration unless confirmed by hidden runtime flow, reflection, batch job, or external plugin.

11. Folder evidence nên tạo

```
migration-evidence/
├── 01-ant-classpath.txt
├── 02-source-imports.txt
├── 02-source-imports-unique.txt
├── 03-jar-package-index.txt
├── 04-config-dependency-usage.txt
├── jdeps/
│   ├── keilib-jdeps.txt
│   ├── servercommon-jdeps.txt
│   └── KEIKA-jdeps.txt
├── report/
└── dependency-usage-matrix.csv
```

12. Script tạo package index từ JAR

Mục tiêu: biết mọi JAR chứa package/class nào. Sau đó map import với JAR tương ứng.

```
mkdir -p evidence

for jar in $(find ServerLib/lib -name "*.jar" | sort); do
    echo "### $jar" >> evidence/03-jar-package-index.txt
    jar tf "$jar" | grep "\.class$" | sed 's|/|.|g' | sed 's|.class$||' | sort >>
evidence/03-jar
-package-index.txt
done
```

Vi dụ map:

```
import com.fasterxml.jackson.databind.ObjectMapper

-> jackson-databind-2.13.1.jar contains com.fasterxml.jackson.databind.ObjectMapper
```

13. Script scan dependency theo module

File goi y: scripts/scan-module-deps.sh

```
#!/usr/bin/env bash
set -euo pipefail

MODULES=("keilib" "servercommon" "KEIKA")
EVIDENCE_DIR="evidence"

mkdir -p "$EVIDENCE_DIR/source" "$EVIDENCE_DIR/config" "$EVIDENCE_DIR/jdeps"

echo "Scanning source imports..."

for module in "${MODULES[@]}; do
  if [ -d "$module" ]; then
    find "$module" -name "*.java" -exec grep -H "^import " {} \; | sort -u >
"$EVIDENCE_DIR/source/{module}-imports.txt" || true

    find "$module" \( -name "*.xml" -o -name "*.properties" -o -name "*.jsp" -o -name "*.tag" -o
-name "*.tld" -o -name "*.xhtml" \) -exec grep -HnE "org\.springframework|org\.terasoluna|org\.apache\.ibatis|
com\.fasterxml\.jackson|javax\.servlet|org\.apache\.tiles|org\.quartz|org\.dozer|joda|logback|slf4j" {} \; >
"$EVIDENCE_DIR/config/{module}-config-usage.txt" || true
  fi
done

echo "Done. Evidence generated under $EVIDENCE_DIR"
```

14. Wording de noi voi customer / QA

Nen noi theo huong risk-based migration, khong noi kieu dependency nao do chac chan khong dung neu chua co du coverage.

We should not assume that all JARs listed in all.userlibraries are required application dependencies. The file appears to be an Eclipse IDE library configuration and includes runtime, test, extension, server API, source JAR references, and possibly historical libraries.

To reduce migration scope safely, we will perform dependency usage analysis using:

1. Actual Ant build classpath from KEIKA/build.xml
2. Java source import scan per module
3. Bytecode-level analysis using jdeps
4. XML/JSP/properties configuration scan
5. Runtime loaded-class evidence from smoke/integration tests

Based on the evidence, dependencies will be classified as:

- directly used
- config/runtime used
- transitive required
- test-only
- server-provided
- no evidence found / candidate for exclusion

Only dependencies with direct, config, bytecode, runtime, or transitive evidence should be included in the migration scope.

15. Doan ngan de hoi khach hang / senior

Hi team, to prepare QA evidence and reduce unnecessary dependency migration scope, I would like to perform a dependency usage analysis instead of assuming every JAR in all.userlibraries must be migrated.

Since modules like keilib and servercommon do not have their own build files and appear to be built through KEIKA,

I will first capture the actual KEIKA Ant build classpath. Then I will scan source imports, compiled bytecode with jdeps, XML/JSP/properties configuration, and runtime loaded classes.

The goal is to classify each library as directly used, config/runtime used, transitive required, test-only, server-provided, or no evidence found. This will help us justify which dependencies must be migrated and which can be excluded or left out of scope.

Could you please confirm whether this approach is acceptable for QA evidence?

16. Ket luan thuc te cho case hien tai

Khong nen bat dau bang viec khai bao Ivy dependency per module ngay. Thu tu an toan hon la:

1. Treat all.userlibraries as candidate list only.
2. Extract actual KEIKA Ant classpath.
3. Scan keilib/servercommon/KEIKA source imports.
4. Run jdeps on compiled output per module.
5. Scan XML/JSP/properties.
6. Capture runtime loaded JARs from smoke test.
7. Produce dependency usage matrix.
8. Chi migrate nhom co evidence.

Final QA statement

all.userlibraries shows what is available to Eclipse, but not necessarily what each module actually uses. The migration scope should be based on actual Ant build classpath plus source, bytecode, configuration, and runtime evidence.

17. Appendix - Checklist de thuc hien

Step	Task	Output	Priority
1	Lay Ant classpath	ant print-classpath > evidence/01-ant-classpath.txt	Must have
2	Scan Java import	grep import theo module	Must have
3	Scan compiled bytecode	jdeps tren build/classes hoac target/classes	Must have neu co compiled output
4	Scan config/JSP/XML	grep dependency package patterns	Must have cho Spring/Terasoluna legacy
5	Capture runtime loaded classes	-verbose:class hoac -Xlog:class+load	Strong evidence, phu thuoc test coverage

6	Generate usage matrix	CSV/table cho QA	Must have
7	Classify action	Migrate / test-only / provided / exclude candidate	Must have